

Assessly

by YURank

Özgür Can Güngör
İrem Şura Erkan
Bermal Deniz Akkaya
Berkan Mursal

Personas:

Can Demir – Second-Year Computer Engineering Student

Can Demir, age 20, is a second-year Computer Engineering student. He moved from Ankara to İzmir for university and is currently focused on improving his algorithmic thinking skills. Can practices coding regularly but sometimes struggles with efficiency optimization. In traditional grading systems, he only receives a numeric score without detailed feedback, which makes it difficult to understand where he lost points.

He values:

- Immediate feedback

- Clear test case results

- Transparency in grading

- Equal evaluation for all students

Can wants a system that rewards effort and understanding rather than shortcuts.

Assessly helps him learn through structured feedback and performance analysis.

Dr. Suphi Uçar – Senior Programming Instructor

Dr. Suphi Uçar, age 45, is a senior lecturer in the Department of Computer Engineering. He has been studying Algorithms, Image Processing, and Software Engineering Fundamentals for more than 20 years.

Each semester, he manages over 120 students and assigns weekly programming tasks through Moodle. While he has strong expertise in evaluating algorithmic correctness, the manual grading process has become increasingly time-consuming. Reviewing code line-by-line, checking efficiency, and verifying originality takes hours.

Recently, the rise of AI tools such as ChatGPT has introduced a new concern: distinguishing between genuine student effort and AI-generated solutions. Dr. Uçar values fairness and academic integrity deeply and believes that engineering education must prioritize originality and critical thinking.

He expects a system that can:

- Automate correctness evaluation

- Detect plagiarism reliably

- Estimate AI-generated code probability

- Provide transparent grading reports

- Integrate seamlessly with Moodle

Assessly allows him to focus on teaching rather than repetitive grading tasks..

Murat Kaya – University IT Administrator

Murat Kaya, age 32, is responsible for maintaining the university's Moodle infrastructure and server systems. He has a background in Information Systems and manages LMS integration and database operations.

His primary concerns are:

System security

Data privacy

Server scalability

Secure sandbox execution

API reliability

He needs a system that integrates smoothly without overloading university servers.

Assessly's Docker-based execution and API-based LMS integration meet his technical requirements.

Senario:

It is the submission deadline for Dr. Suphi Uçar's programming assignment. More than 120 students have uploaded their solutions to Moodle. The task requires implementing Merge Sort with strict time complexity constraints.

In previous semesters, Dr. Uçar spent days manually reviewing submissions, checking correctness, comparing similarities, and identifying possible AI-generated code. The process was slow and exhausting.

This semester, Assessly is integrated into Moodle.

Immediately after the deadline, the system executes each submission in a secure sandbox environment. It runs predefined test cases, measures runtime and memory usage, analyzes structural similarity across students, and calculates AI-generation probability.

Within minutes, Dr. Uçar receives a complete evaluation dashboard including final grades, similarity percentages, AI risk indicators, and performance metrics.

Meanwhile, Can Demir logs in to see detailed feedback instead of just a numeric score. He can clearly identify failed test cases and areas for improvement.

At the same time, Murat Kaya monitors the system infrastructure. All submissions are isolated securely, and grades are synchronized automatically with Moodle without affecting server stability.

What previously required days of manual work is now completed in minutes — fairly, securely, and transparently.

User Stories:

- As a student, I want to upload different types of assignments so that all my coursework can be evaluated through a single system.
- As a student, I want to see how well my submission matches the assignment prompt so that I know if I met the expectations.
- As a student, I want to see which test cases passed or failed so that I can understand my mistakes.
- As a student, I want to write my code directly in a web-based editor so that I don't have to install an IDE or compiler on my computer .
- As an instructor, I want to detect plagiarism and AI-generated content so that academic integrity is preserved.
- As an instructor, I want to quickly access only flagged submissions so that my attention is focused where it is most needed.
- As an instructor, I want to create a new programming assignment with a title, description, and deadline so that students clearly know what they need to do .
- As an instructor, I want to upload programming assignments to the system so that students can submit their code digitally.
- As an instructor, I want to define test cases and grading percentages so that the system can evaluate submissions automatically and fairly.
- As an IT administrator, I want the system to run submissions in a secure sandbox environment so that the university servers are protected.
- As an administrator, I want Assessly to integrate with Moodle so that grades are synchronized automatically.
- As a system administrator, I want to execute the code submitted by students in isolated environments so that code entering an infinite loop or containing malicious intent cannot crash the main server .

Feature List:

Feature 1: Assignment & Course Management

This feature serves as the foundation of the system by allowing instructors to create and manage programming assignments within the platform.

- Feature Description:

A module that enables instructors to create assignments, upload problem descriptions, define constraints, add test cases, and set grading weights. The system integrates directly with Moodle to synchronize submissions and grades.

- Associated User Stories:

- o As an instructor, I want to upload programming assignments so that students can submit their solutions digitally.
- o As an instructor, I want to define grading weights and test cases so that evaluation is automated and consistent.
- o As an administrator, I want Moodle integration so that grades are synchronized automatically.

Feature 2: Secure Code Execution Engine

This feature represents the core technical functionality of the platform by ensuring safe and controlled execution of student submissions.

- Feature Description:

A Docker-based sandbox environment that compiles and executes submitted code under defined runtime and memory limits. Each submission is isolated to protect system infrastructure.

- Associated User Stories:

- o As a student, I want my code to be executed automatically so that I can see if it works correctly.
- o As an instructor, I want runtime and memory limits enforced so that inefficient solutions are identified fairly.
- o As an administrator, I want each submission to run in an isolated environment so that the system remains secure.

Feature 3: Automated Test Case Evaluation

This feature ensures objective and consistent grading based on predefined correctness criteria.

- Feature Description:

An automated testing module that runs visible and hidden test cases, compares outputs with expected results, and calculates a correctness score for each submission.

- Associated User Stories:

- o As a student, I want to see which test cases passed or failed so that I can understand my mistakes.

- o As an instructor, I want hidden test cases so that students cannot hard-code answers.
- o As a student, I want fast evaluation results so that I do not need to wait long for feedback.

Feature 4: AI-Based Code Analysis

This feature addresses the increasing use of AI tools in academic programming assignments.

- Feature Description:

An AI-supported analysis engine that evaluates semantic alignment between the assignment prompt and submitted code, and calculates an AI-generation probability score based on structural patterns.

- Associated User Stories:

- o As an instructor, I want to see an AI probability score so that I can review suspicious submissions more carefully.
- o As an instructor, I want AI analysis to support my decision-making process rather than replace it.
- o As a student, I want my submission evaluated fairly based on its relevance to the assignment prompt.

Feature 5: Plagiarism Detection System

This feature protects academic integrity by identifying structural similarities between submissions.

- Feature Description:

A similarity detection module that compares submissions using token-based comparison and Abstract Syntax Tree (AST) analysis to identify potential copying.

- Associated User Stories:

- o As an instructor, I want to detect similarity between student submissions so that copying can be identified quickly.
- o As an instructor, I want similarity percentages displayed clearly so that I can focus on high-risk cases.
- o As a student, I want fair grading so that copied solutions do not negatively impact honest students.

Feature 6: Smart Grading & Feedback System

This feature combines all evaluation components into a transparent final result.

- Feature Description:

A weighted scoring system that calculates final grades based on correctness, efficiency, plagiarism, and AI analysis. The system generates structured feedback explaining the grading breakdown.

- Associated User Stories:

- o As a student, I want to understand how my final grade is calculated so that the evaluation process is transparent.

- o As a student, I want detailed feedback so that I can improve in future assignments.

- o As an instructor, I want a summarized evaluation report so that I can review results efficiently.